Franchise Expansion Platform

A multi-brand web platform that tells any brand where to open franchises in India, keeps that plan current as laws, taxes, markets and consumer behaviour change, and takes verified franchise applications for the locations the brand chooses. Product spec: `docs/SPEC.md`. Read the relevant section before planning any task.

Current priority: hackathon Case 1 (SPEC §12)

Build the Case 1 MVP first: judges upload an Excel workbook (applications, territories, existing stores), get an explainable decision per applicant with territory allocation, ask follow-up questions, change an application's facts and see it recomputed, and get onboarding plans. It runs at a public URL with no login. Flows A and B and live verification are roadmap work. Don't build them during the hackathon slices unless the slice prompt says so. Case 1 sources: `docs/case1-brief.pdf` (the brief) and `data/case1-template.xlsx` (the workbook). If either is missing, say so and stop; never invent sheet or column names.

Three flows (SPEC §4–§6)

- **A · Recommend, then refine:** brand profile → every PIN code scored (demand, consumer spending, Engel's Law fit, competition, cost, cannibalisation) → Gemini explains → brand adds inputs → re-run.
- **B · Change Radar:** daily scan of government, consumer-spending and market sources → Gemini rates impact on each brand goal → affected locations re-scored → Change Impact Score.
- **C · Applications and verification:** brand opens a location (or uploads applications in bulk) → applicant applies with consent → identity via a licensed DigiLocker vendor, finances via a regulated lender partner → screening and territory allocation → the brand decides.

Non-negotiable rules

1. **The brand decides; the platform recommends.** No code path approves a partner, signs anything, or changes a brand's plan on its own. Screening outputs are recommendations; only a brand action makes a partner "Approved". Change Radar proposes re-scores; the brand accepts them.
2. **Numbers come from code, never from Gemini.** Scores, rankings, distances, spend figures and payback come from the scoring engine. Gemini explains, classifies, extracts and drafts. `numberGuard` (SPEC §8) rejects Gemini output containing a number that is neither in the computed inputs it was given (after normalisation) nor in the task's allowlist.
3. **No earnings promises to applicants.** Payback and revenue estimates are internal to the brand, always shown as a labelled range with written assumptions. Applicant-facing text never states expected earnings.
4. **Never ask applicants for money.** No payment flows for applicants anywhere. Every applicant message carries the fraud notice from SPEC §9.

5. **Consent before data.** Every verification (DigiLocker, lender partner, bank account check) starts with an explicit, purpose-specific consent record. No consent, no call. Comply with the DPDP Act: purpose limitation, retention limits, deletion on request and for rejected applicants after `RETENTION_DAYS` (value set in config; proposed in `docs/DESIGN.md`, needs approval).
6. **The platform never holds raw bank statements.** Financial checks go through a regulated lender partner, who returns an attestation ("funds verified ≥ ₹X", "loan sanctioned ₹Y"). Store the attestation, not the data. Investment capacity typed by an applicant is a claim, never verified funds.
7. **Fairness.** Name, gender, religion, caste, age and language never enter scoring or eligibility. Language only shapes communication and training.
8. **Google Maps terms.** Store only Google place IDs indefinitely; lat/lng from Google APIs at most 30 days; never build a database of Google places, reviews or addresses. Base geography (including PIN centroids for distance rules) comes from open data (SPEC §7).
9. **Cite sources.** Every Change Radar item and every consumer-behaviour figure carries its source, date and geographic level. Estimates finer than the source's level are labelled as estimates.
10. **Guardrail rules are deterministic.** One store per PIN, distance from existing stores, territory slots, verification gates and eligibility minimums are code with tests, configurable per brand, never prompts. Allocation never depends on input row order.
11. **Isolation.** A brand never sees another brand's data, stores, applicants or results; enforce in the database (row-level security) and test it. In the public hackathon demo, each browser session gets its own throwaway workspace; one session never sees another's uploads or edits.
12. **Changes go through the record.** A follow-up ("APP-012 found a ground-floor space") becomes a proposed field change that the user confirms; code then recomputes. Gemini never edits a record or a decision directly.

Gemini

- Use the Google Gen AI SDK. Model IDs live in config (`GEMINI_MODEL_FAST`, `GEMINI_MODEL_REASONING`); at kickoff, check the current stable models at ai.google.dev/gemini-api/docs/models, record the IDs and the date checked in `docs/DESIGN.md`, and pin explicit stable versions. No preview or retired models; never rely on an auto-updating alias where evals depend on behaviour.
- Use structured output (JSON schema) for every machine-read response; validate and retry once, then fail safely (show the computed result without the AI text).
- Grounding with Google Search for Change Radar research; grounding with Google Maps only for user-facing place context. Keep returned source links and show them. Check whether the pinned model supports grounding and a response schema in one call; if not, ground first, then structure in a second call.
- Prompts live in `prompts/` as versioned files with eval cases next to them.

Frontend design

- Before any UI work, read the design skill files in `.claude/skills/` and follow them. Do not invent a visual style, palette or type scale; tokens come from the skills.

- Maps: Google Maps JavaScript API with the brand's ranked PINs, existing stores, cannibalisation radius and competitor layer. The map must stay usable on mobile.

Context budget

- Send Gemini only what a task needs: the computed scores and factors for the PINs or applicants in question, the brand profile, and the relevant source excerpts. Never whole datasets or whole workbooks.
- Applies to you too: read files and data samples when a task needs them.

Stack

- `apps/web`: Next.js + TypeScript, styled per the design skills; Google Maps JS API.
- `services/engine`: Python (FastAPI) for ingestion, scoring, allocation, screening, demand model and Change Radar.
- `db/`: Postgres (+ PostGIS when Flow A needs it), migrations, row-level security.
- `jobs/`: scheduler and queue (daily Change Radar, re-scores, retention deletes).
- `prompts/`, `evals/`, `data/` (open datasets, loaders and non-sensitive fixtures; no proprietary or real applicant data in git).
- Keep the hackathon deployment as small as the acceptance criteria allow; justify any extra service in `docs/DESIGN.md`.
- **Built so far:** `app/` is the Case 1 MVP as a static page; all logic is in `app/engine.js` and runs in the browser (no server, so each tab is its own isolated session). Tests: `cd app && node --test`. The stack above is for the platform slices (P1 onward).

Commands

- `make setup` · `make dev` · `make test` · `make lint` · `make typecheck`
- `make eval` — scoring, Gemini, Change Radar and screening evals
- `make data` — refresh open datasets

Working rules

- One slice at a time, in the order in `docs/SPEC.md` §12 and `docs/PROMPTS.md`. List out-of-scope needs instead of doing them.
- Plan first; wait for approval on schema changes, scoring formulas, eligibility thresholds, guardrail rules, consent flows and any third-party integration.
- All third-party integrations (DigiLocker vendor, lender partner, bank account check, Maps, Gemini) sit behind interfaces with sandbox and mock adapters; tests never call live services.
- Never commit secrets or applicant data. Keep `.env.example` current.
- Record decisions and open assumptions in `docs/DESIGN.md`.
- Before finishing: tests, lint, typecheck, evals, and a list of what to check by hand in the UI. A feature isn't done because the UI shows it; test the state change underneath.